



UNIT NO: 3

Greedy Techniques and Dynamic Programming

T.Y (Semester - V)
2024-25





UNIT 3 Contents ->

Greedy strategy: Principle, algorithms: greedy knapsack problem, scheduling algorithm- activity selection problem

Dynamic Programming: Principle, algorithms: 0/1 knapsack problem, the traveling salespersons problem, Warshall's and Floyd's Algorithms

Now

- **Greedy strategy:** Introduction, Principle
- **Algorithms:** greedy knapsack problem
- **Scheduling algorithm-** activity selection problem



Greedy strategy

- Among all the algorithmic approaches, the simplest and straightforward approach is the Greedy method.
- We encounter various types of computational problems each of which requires a different technique for solving.
- A problem that requires a maximum or minimum result is the optimization problem.
- There are broadly 3 ways to solve optimization problems:
 1. The greedy method
 2. Dynamic programming
 3. Branch and bound technique



Greedy strategy

- A greedy algorithm is an algorithmic paradigm that follows the problem-solving heuristic of making the locally optimal choice at each stage with the hope of finding a global optimum.
- In other words, a greedy algorithm chooses the best possible option at each step, without considering the consequences of that choice on future steps.
- The greedy algorithm is useful when a problem can be divided into smaller sub problems, and the solution of each sub problem can be combined to solve the overall problem.
- A classic example of a problem that can be solved using a greedy algorithm is the “coin change” problem.
- The problem is to make change for a given amount of money using the least possible number of coins.



- A greedy problem statement follows two properties mentioned below:

1) Greedy Choice Property: Choosing the best option at each phase can lead to a global (overall) optimal solution.

2) Optimal Substructure: If an optimal solution to the complete problem contains the optimal solutions to the sub problems, the problem has an optimal substructure.



The general structure of a greedy algorithm can be summarized in the following steps:

1. Identify the problem as an optimization problem where we need to find the best solution among a set of possible solutions.
2. Determine the set of feasible solutions for the problem.
3. Identify the optimal substructure of the problem, meaning that the optimal solution to the problem can be constructed from the optimal solutions of its subproblems.
4. Develop a greedy strategy to construct a feasible solution step by step, making the locally optimal choice at each step.
Prove the correctness of the algorithm by showing that the locally optimal choices at each step lead to a globally optimal solution.



Characteristic components of greedy algorithm:

The feasible solution: A subset of given inputs that satisfies all specified constraints of a problem is known as a “feasible solution”.

Optimal solution: The feasible solution that achieves the desired extremum is called an “optimal solution”. In other words, the feasible solution that either minimizes or maximizes the objective function specified in a problem is known as an “optimal solution”.

Feasibility check: It investigates whether the selected input fulfils all constraints mentioned in a problem or not. If it fulfils all the constraints then it is added to a set of feasible solutions; otherwise, it is rejected.

Optimality check: It investigates whether a selected input produces either a minimum or maximum value of the objective function by fulfilling all the specified constraints. If an element in a solution set produces the desired extremum, then it is added to a selection of optimal solutions.

Optimal substructure property: The globally optimal solution to a problem includes the optimal sub solutions within it.

Greedy choice property: The globally optimal solution is assembled by selecting locally optimal choices. The greedy approach applies some locally optimal criteria to obtain a partial solution that seems to be the best at that moment and then find out the solution for the remaining sub-problem



Counting Coins Problem

The Counting Coins problem is to count to a desired value by choosing the least possible coins and the greedy approach forces the algorithm to pick the largest possible coin. If we are provided coins of 1, 2, 5 and 10 and we are asked to count 18 then the greedy procedure will be –

- 1 – Select one 10 coin, the remaining count is 8
- 2 – Then select one 5 coin, the remaining count is 3
- 3 – Then select one 2 coin, the remaining count is 1
- 4 – And finally, the selection of one 1 coins solves the problem

Though, it seems to be working fine, for this count we need to pick only 4 coins. But if we slightly change the problem then the same approach may not be able to produce the same optimum result.

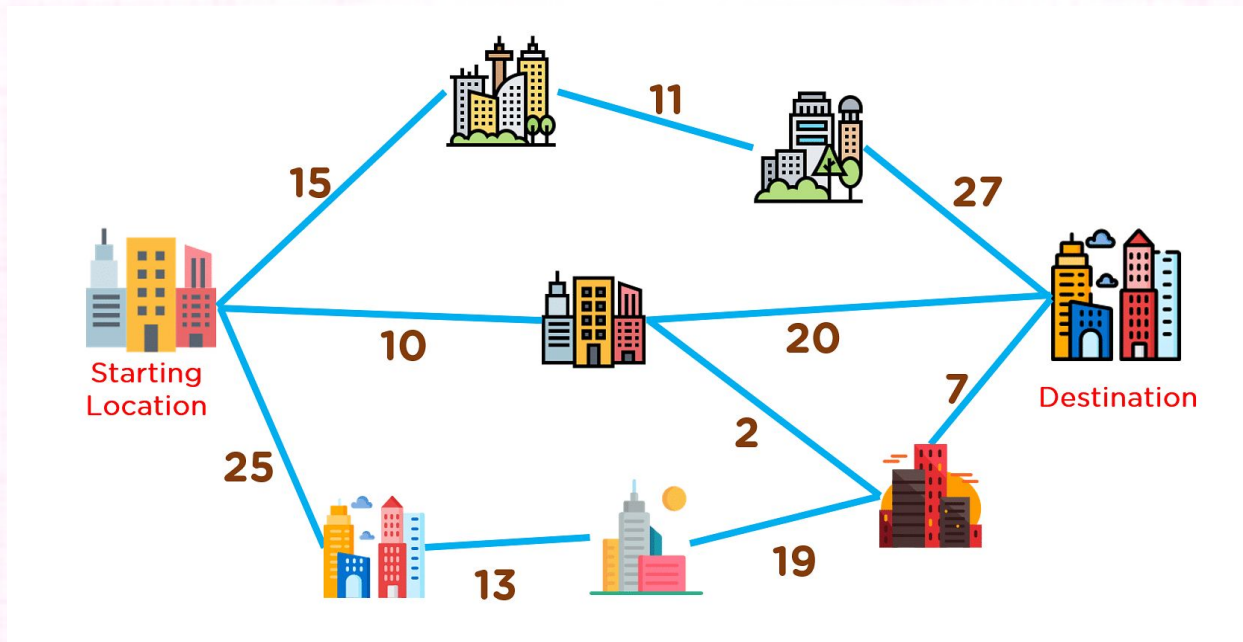
For the currency system, where we have coins of 1, 7, 10 value, counting coins for value 18 will be absolutely optimum but for count like 15, it may use more coins than necessary. For example, the greedy approach will use $10 + 1 + 1 + 1 + 1 + 1$, total 6 coins. Whereas the same problem could be solved by using only 3 coins ($7 + 7 + 1$)

Hence, we may conclude that the greedy approach picks an immediate optimized solution and may fail where global optimization is a major concern.



Example of Greedy Algorithm

Problem Statement: Find the best route to reach the destination city from the given starting point using a greedy method.

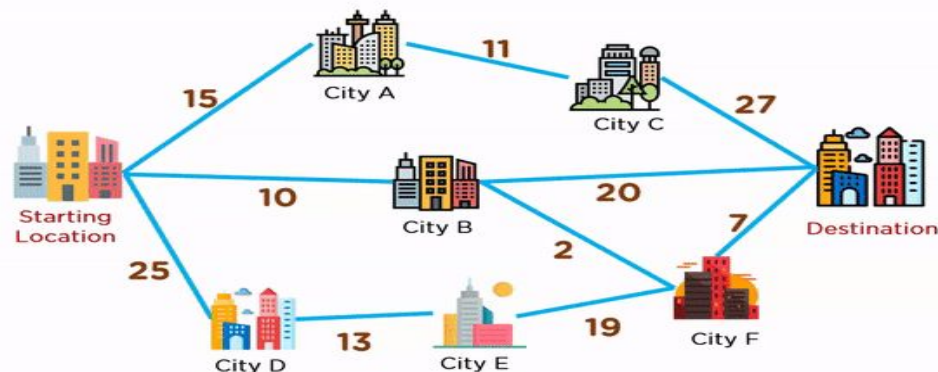




Solution steps :

- Start from the source vertex.
- Pick one vertex at a time with a minimum edge weight (distance) from the source vertex.
- Add the selected vertex to a tree structure if the connecting edge does not form a cycle.
- Keep adding adjacent fringe vertices to the tree until you reach the destination vertex.

The animation given below explains how paths will be picked up in order to reach the destination





Applications of Greedy Algorithms:

- Finding an optimal solution (Activity selection, Fractional Knapsack, Job Sequencing, Huffman Coding).
- Finding close to the optimal solution for NP-Hard problems like TSP.
- Greedy algorithm is used to select the jobs that will be completed before their respective deadlines and maximizes the profit.
- Greedy algorithms are used to cluster data points together based on certain criteria, such as distance or similarity.
- The problem is broken down into smaller subproblems that are solved independently, but many of these subproblems are identical or overlapping.



Advantages of the Greedy Approach:

- The greedy approach is easy to implement.
- Typically have less time complexity.
- Greedy algorithms can be used for optimization purposes or finding close to optimization in case of Hard problems.
- The greedy approach can be very efficient, as it does not require exploring all possible solutions to the problem.
- The solutions to subproblems can be stored in a table, which can be reused for similar problems.



Disadvantages of the Greedy Approach:

- The local optimal solution may not always be globally optimal.
- Lack of proof of optimality.
- The greedy approach is only applicable to problems that have the property of greedy-choice property meaning not all problems can be solved using this approach.
- The greedy approach is not easily adaptable to changing problem conditions.





Greedy Basic algorithm

Algorithm Greedy(a , n)

```
{  
  {  
    for i = 1 to n do  
      x= select(a)  
      if feasible (x) then  
        solution =solution + x ;  
    }  
  }  
}
```

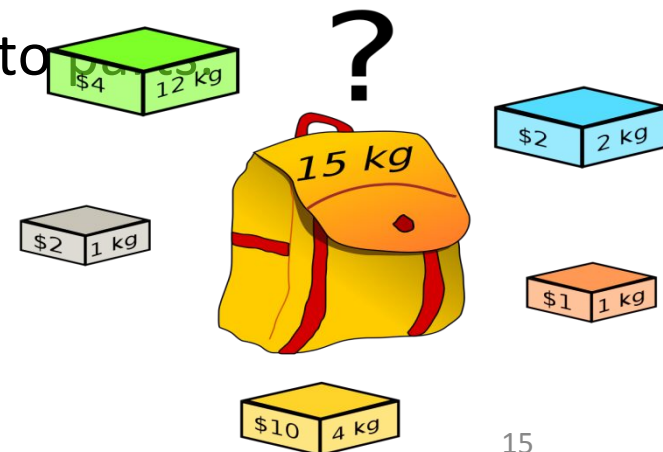



Greedy Knapsack Problem

- The selection of some things, each with profit and weight values, to be packed into one or more knapsacks with capacity is the fundamental idea behind all families of knapsack problems.

- The knapsack problem had two versions that are as follows:
Binary or 0/1 knapsack : Item cannot be broken down into parts.

Fractional knapsack : Item can be divided into parts





Fractional Knapsack Problem

- The fractional knapsack problem is also one of the techniques which are used to solve the knapsack problem.
- In fractional knapsack, the items are broken in order to maximize the profit.
- The problem in which we break the item is known as a Fractional knapsack problem.
- To explain this problem a little easier, consider a test with 12 questions, 10 marks each, out of which only 10 should be attempted to get the maximum mark of 100.
- The test taker now must calculate the highest profitable questions – the one that he's confident in – to achieve the maximum mark.
- However, he cannot attempt all the 12 questions since there will not be any extra marks awarded for those attempted answers.
- This is the most basic real-world application of the knapsack problem.



Knapsack Algorithm

- The weights (W_i) and profit values (P_i) of the items to be added in the knapsack are taken as an input for the fractional knapsack algorithm and the subset of the items added in the knapsack without exceeding the limit and with maximum profit is achieved as the output.

Algorithm

- Consider all the items with their weights and profits mentioned respectively.
- Calculate P_i/W_i of all the items and sort the items in descending order based on their P_i/W_i values.
- Without exceeding the limit, add the items into the knapsack.
- If the knapsack can still store some weight, but the weights of other items exceed the limit, the fractional part of the next time can be added.

- Hence, giving it the name fractional knapsack problem.



Example

For the given set of items and the knapsack capacity of 10 kg, find the subset of the items to be added in the knapsack such that the profit is maximum.

Items	1	2	3	4	5
Weights (in kg)	3	3	2	5	1
Profits	10	15	10	12	8

Solution

Step 1

Given, $n = 5$

$W_i = \{3, 3, 2, 5, 1\}$

$P_i = \{10, 15, 10, 12, 8\}$

Calculate P_i/W_i for all the items

Items	1	2	3	4	5
Weights (in kg)	3	3	2	5	1
Profits	10	15	10	20	8
P_i/W_i	3.3	5	5	4	8

Step 2

Arrange all the items in descending order based on P_i/W_i

Items	5	2	3	4	1
Weights (in kg)	1	3	2	5	3
Profits	8	15	10	20	10
P_i/W_i	8	5	5	4	3.3



Step 3

- Without exceeding the knapsack capacity, insert the items in the knapsack with maximum profit.
- Knapsack = {5, 2, 3}
- However, the knapsack can still hold 4 kg weight, but the next item having 5 kg weight will exceed the capacity.
- Therefore, only 4 kg weight of the 5 kg will be added in the knapsack.

Items	5	2	3	4	1
Weights (in kg)	1	3	2	5	3
Profits	8	15	10	20	10
Knapsack	1	1	1	4/5	0

Hence, the knapsack holds the weights = $[(1 * 1) + (1 * 3) + (1 * 2) + (4/5 * 5)] = 10$, with maximum profit of $[(1 * 8) + (1 * 15) + (1 * 10) + (4/5 * 20)] = 37$.



Algorithm GREEDY_FRACTIONAL_KNAPSACK(X, V, W, M)

// Description : Solve the knapsack problem using greedy approach

// Input: X : An array of n items

- V : An array of profit associated with each item
- W : An array of weight associated with each item
- M : Capacity of knapsack
- Output : SW : Weight of selected items SP : Profit of selected items
- // Items are presorted in decreasing order of $p_i = v_i / w_i$ ratio
- $S \leftarrow \Phi$ // Set of selected items,
- initially empty $SW \leftarrow 0$ // weight of selected items
- $SP \leftarrow 0$ // profit of selected items
- $i \leftarrow 1$ **while** $i \leq n$
- **do if** $(SW + w[i]) \leq M$ **then**
- $S \leftarrow S \cup X[i]$ $SW \leftarrow SW + W[i]$ $SP \leftarrow SP + V[i]$
- **else**
- $frac \leftarrow (M - SW) / W[i]$ $S \leftarrow S \cup X[i] * frac$ // Add fraction of item
- $X[i]$ $SP \leftarrow SP + V[i] * frac$ // Add fraction of profit
- $SW \leftarrow SW + W[i] * frac$ // Add fraction of weight
- **End**
- $i \leftarrow i + 1$ **end**



Scheduling Algorithm- Activity Selection Problem

Activity Selection problem is a approach of selecting non-conflicting tasks based on start and end time and can be solved in $O(N \log N)$ time using a simple greedy approach.

The problem statement for **Activity Selection** is that "Given a set of n activities with their start and finish times, we need to select **maximum number of non-conflicting activities** that can be performed by a single person, given that the person can handle only one activity at a time.

" The Activity Selection problem follows **Greedy approach**.

i.e. at every step, we can make a choice that looks best at the moment to get the optimal solution of the complete problem.



- The Activity selection problem can be solved using **Greedy Approach**.
- Our task is to maximize the number of non-conflicting activities.
- Two activities **A1** and **A2** are said to be non-conflicting if $S1 \geq F2$ or $S2 \geq F1$,
- where **S** and **F** denote the start and end time respectively.





Example

	0	1	2	3	4	5	6
START	1	3	2	0	5	8	11
END	3	4	5	7	9	10	12

SELECTED

START	1	3	2	0	5	8	11
END	3	4	5	7	9	10	12

$START[1] \geq END[0]$, SELECTED

START	1	3	2	0	5	8	11
END	3	4	5	7	9	10	12

$START[2] < END[1]$, REJECTED

START	1	3	2	0	5	8	11
END	3	4	5	7	9	10	12

$START[3] < END[1]$, REJECTED



START	1	3	2	0	5	8	11
END	3	4	5	7	9	10	12

START[4] ≥ END[2], SELECTED

START	1	3	2	0	5	8	11
END	3	4	5	7	9	10	12

START[5] < END[4], REJECTED

START	1	3	2	0	5	8	11
END	3	4	5	7	9	10	12

START[6] ≥ END[4], SELECTED



In this example, we take the start and finish time of activities as follows:

start = [1, 3, 2, 0, 5, 8, 11]

finish = [3, 4, 5, 7, 9, 10, 12]

- 1) Sorted by their finish time, the activity 0 gets selected.
- 2) As the activity 1 has starting time which is equal to the finish time of activity 0, it gets selected.
- 3) Activities 2 and 3 have smaller starting time than finish time of activity 1, so they get rejected.
- 4) Based on similar comparisons, activities 4 and 6 also get selected, whereas activity 5 gets rejected.
- 5) In this example, in all the activities 0, 1, 4 and 6 get selected, while others get rejected.



Dynamic Programming

- Dynamic programming approach is similar to divide and conquer in breaking down the problem into smaller and yet smaller possible sub-problems.
- But unlike divide and conquer, these sub-problems are not solved independently.
- Rather, results of these smaller sub-problems are remembered and used for similar or overlapping sub-problems.
- The main use of dynamic programming is to solve optimization problems.
- Here, optimization problems mean that when we are trying to find out the minimum or the maximum solution of a problem.
- The dynamic programming guarantees to find the optimal solution of a problem if the solution exists.





The following are the steps that the dynamic programming follows:

- It breaks down the complex problem into simpler subproblems.
- It finds the optimal solution to these sub-problems.
- It stores the results of subproblems (memoization). The process of storing the results of subproblems is known as memorization.
- It reuses them so that same sub-problem is calculated more than once.
- Finally, calculate the result of the complex problem.





The General Method

Dynamic programming

An algorithm design method that can be used when the solution can be viewed as the result of a sequence of decisions

Some solvable by Greedy method under the condition

Condition: an optimal sequence of decisions can be found by making the decisions one at a time and never making an erroneous decision

For many other problems

Not possible to make stepwise decisions (based only on local information) in a manner like Greedy method



The General Method

Enumeration vs. dynamic programming

Enumeration

Enumerating all possible decision sequences and picking out the best ? prohibitive time and storage requirements

Dynamic programming

Drastically reducing the time and storage by avoiding some decision sequences that cannot possibly be optimal

Making explicit appeal to the *principle of optimality*

Definition [Principle of optimality] The principle of optimality states that an optimal sequence of decisions has the property that whatever the initial state and decision are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision



The General Method

Greedy method vs. dynamic programming

Greedy method

Only one decision sequence is ever generated

Dynamic programming

Many decision sequences may be generated

But sequences containing suboptimal subsequences discarded because they cannot be optimal due to the principle of optimality



Steps for Dynamic Programming

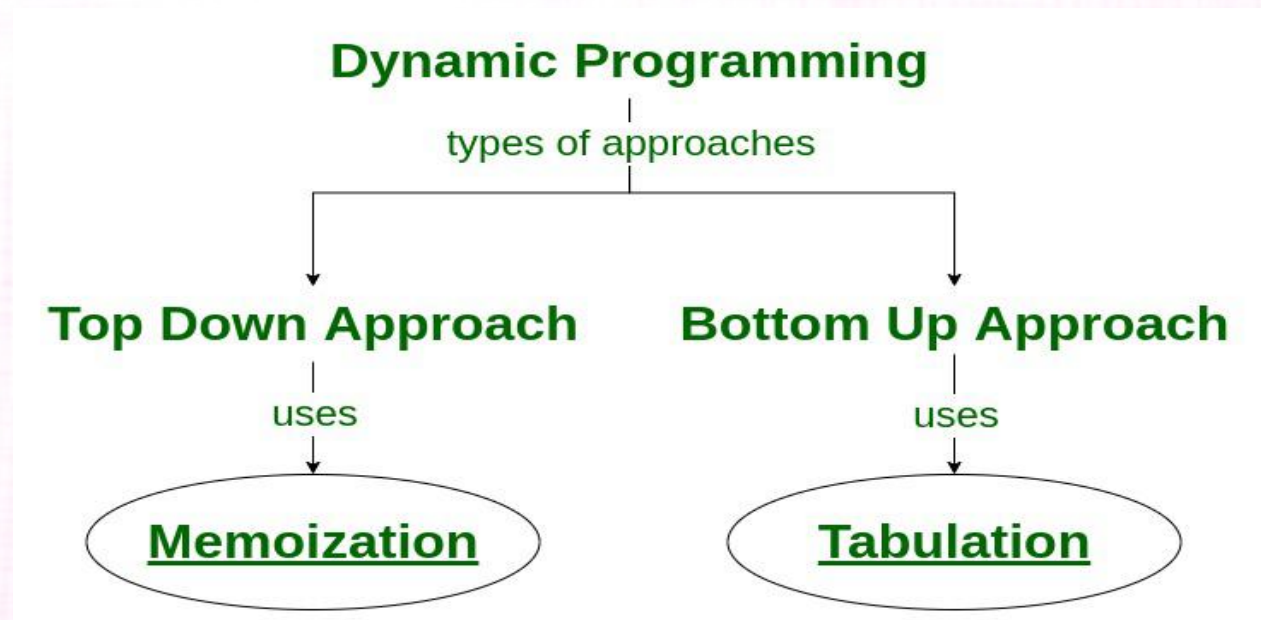
- 1) The problem can be divided into stages with decision required at each stage
- 2) Each state has a number of states associated with it
- 3) The decision at one stage transform one stage to next
- 4) Given current state the optimal decision for each of the remaining state does not depend on the previous state or decision
- 5) There exists a recursive relationship that identifies the optimal solution for stage j , given the stage $j+1$ has already been solved
- 6) One final stage must be solvable by itself.



Approaches of Dynamic Programming :

There are two approaches to dynamic programming:

- Top-down approach
- Bottom-up approach





1)Top-down approach

The top-down approach follows the memorization technique, while bottom-up approach follows the tabulation method.

Here memorization is equal to the sum of recursion and caching.

Recursion means calling the function itself, while caching means storing the intermediate results.

Advantages

It is very easy to understand and implement.

It solves the subproblems only when it is required.

It is easy to debug.

Disadvantages

It uses the recursion technique that occupies more memory in the call stack. Sometimes when the recursion is too deep, the stack overflow condition will occur.

It occupies more memory that degrades the overall performance.



2)Bottom-Up approach

- The bottom-up approach is also one of the techniques which can be used to implement the dynamic programming.
- It uses the tabulation technique to implement the dynamic programming approach.
- It solves the same kind of problems but it removes the recursion.
- If we remove the recursion, there is no stack overflow issue and no overhead of the recursive functions.
- In this tabulation technique, we solve the problems and store the results in a matrix.



Algorithms: 0/1 Knapsack Problem using Dynamic Programming

Algorithm DP_BINARY_KNAPSACK (V, W, M) // Description: Solve binary knapsack problem using dynamic programming
// Input: Set of items X , set of weight W , profit of items V and knapsack capacity M
// Output: Array V , which holds the solution of problem
for $i \leftarrow 1$ to n **do** $V[i, 0] \leftarrow 0$ **end**

for $i \leftarrow 1$ to M **do** $V[0, i] \leftarrow 0$ **end for** $V[0, i] \leftarrow 0$

do for $j \leftarrow 0$ to M **do if** $w[i] \leq j$ **then** $V[i, j] \leftarrow \max\{V[i-1, j], v[i] + V[i-1, j - w[i]]\}$ **else** $V[i, j] \leftarrow V[i-1, j]$ // $w[i] > j$ **end**

end



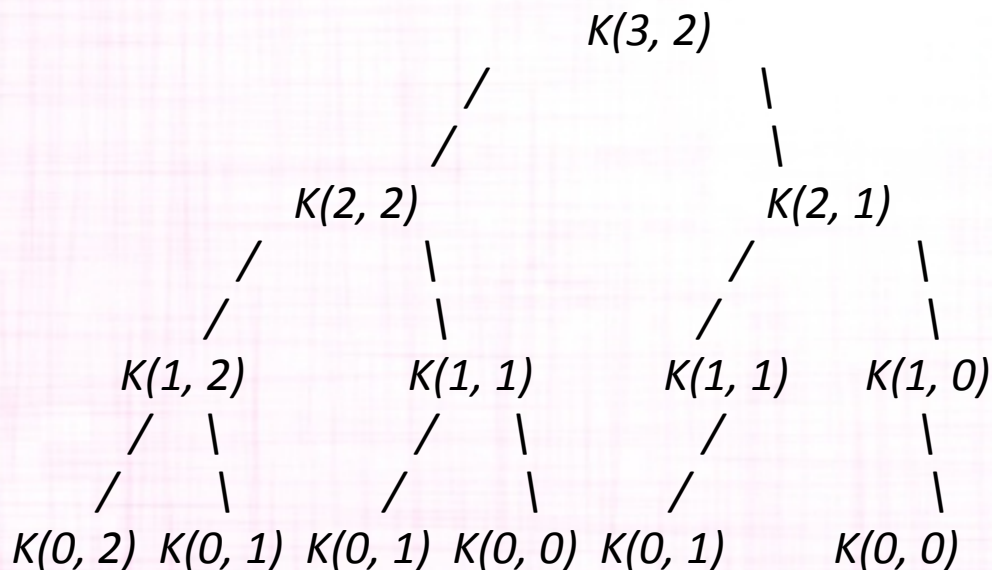
Dynamic Programming Approach for 0/1 Knapsack Problem

Memoization Approach for 0/1 Knapsack Problem using Dynamic Programming:

In the following recursion tree, $K()$ refers to $\text{knapSack}()$. The two parameters indicated in the following recursion tree are n and W .

The recursion tree is for following sample inputs.

$\text{weight[]} = \{1, 1, 1\}$, $W = 2$, $\text{profit[]} = \{10, 20, 30\}$



Recursion tree for Knapsack capacity 2 units and 3 items of 1 unit weight.



Example :-

A thief is robbing a store and can carry a maximal weight of **W** into his knapsack. There are **n** items and weight of i^{th} item is w_i and the profit of selecting this item is **p_i** . What items should the thief take?

▪The algorithm takes the following inputs

- 1)The maximum weight **W**
- 2)The number of items **n**
- 3)The two sequences **$v = \langle v_1, v_2, \dots, v_n \rangle$ and $w = \langle w_1, w_2, \dots, w_n \rangle$**

The set of items to take can be deduced from the table, starting at **$c[n, w]$** and tracing backwards where the optimal values came from.



Let us consider that the capacity of the knapsack is $W = 8$ and the items are as shown in the following table

Item	A	B	C	D
Profit	2	4	7	10
Weight	1	3	5	7





Step 1

Construct an adjacency table with maximum weight of knapsack as rows and items with respective weights and profits as columns.

Values to be stored in the table are cumulative profits of the items whose weights do not exceed the maximum weight of the knapsack (designated values of each row)

← Maximum Weights →

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (1, 2)	0								
2 (3,4)	0								
3 (5,7)	0								
4 (7,10)	0								

Items with Weights and Profits



2) The remaining values are filled with the maximum profit achievable with respect to the items and weight per column that can be stored in the knapsack.

The formula to store the profit values is –

$$c[i,w]=\max\{c[i-1,w-w[i]]+P[i]\}$$

		← Maximum Weights →								
Items with Weights and Profits		0	1	2	3	4	5	6	7	8
	0	0	0	0	0	0	0	0	0	0
	1 (1, 2)	0	1	1	1	1	1	1	1	1
	2 (3,4)	0	1	1	4	6	6	6	6	6
	3 (5,7)	0	1	1	4	6	7	9	9	11
	4 (7,10)	0	1	1	4	6	7	9	10	12

Design and Analysis of Algorithm

Unit 3



To find the items to be added in the knapsack, recognize the maximum profit from the table and identify the items that make up the profit, in this example, its {1, 7}.

		← Maximum Weights →								
Items with Weights and Profits		0	1	2	3	4	5	6	7	8
	0	0	0	0	0	0	0	0	0	0
	1 (1, 2)	0	1	1	1	1	1	1	1	1
	2 (3,4)	0	1	1	4	6	6	6	6	6
	3 (5,7)	0	1	1	4	6	7	9	9	11
	4 (7,10)	0	1	1	4	6	7	9	10	12

The optimal solution is {1, 7} with the maximum profit is 12.



■ The Traveling Salespersons Problem

The travelling salesman problem is a graph computational problem where the salesman needs to visit all cities (represented using nodes in a graph) in a list just once and the distances (represented using edges in the graph) between all these cities are known.

The solution that is needed to be found for this problem is the shortest possible route in which the salesman visits all the cities and returns to the origin city.



■Algorithm :-

- 1) Travelling salesman problem takes a graph $G \{V, E\}$ as an input and declare another graph as the output (say G') which will record the path the salesman is going to take from one node to another.
- 2) The algorithm begins by sorting all the edges in the input graph G from the least distance to the largest distance.
- 3) The first edge selected is the edge with least distance, and one of the two vertices (say A and B) being the origin node (say A).
- 4) Then among the adjacent edges of the node other than the origin node (B), find the least cost edge and add it onto the output graph.
- 5) Continue the process with further nodes making sure there are no cycles in the output graph and the path reaches back to the origin node A .
- 6) However, if the origin is mentioned in the given problem, then the solution must always start from that node only.



Example–

Let's assume S is the subset of cities and belongs to $\{1, 2, 3, \dots, n\}$ where $1, 2, 3 \dots n$ are the cities and i, j are two cities in that subset.

Now $\text{cost}(i, S, j)$ is defined in such a way as the length of the shortest path visiting node in S , which is exactly once having the starting and ending point as i and j respectively.

The dynamic programming algorithm would be:

Set $\text{cost}(i, , i) = 0$, which means we start and end at i , and the cost is 0.

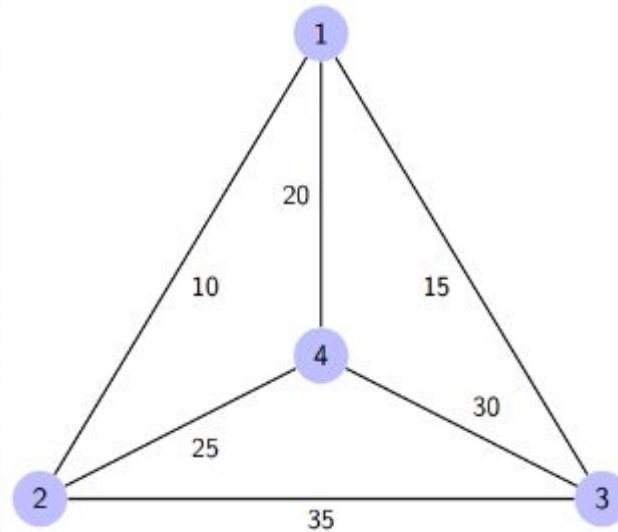
When $|S| > 1$, we define $\text{cost}(i, S, 1) = \infty$ where $i \neq 1$.

Because initially, we do not know the exact cost to reach city i to city 1 through other cities.

Now, we need to start at 1 and complete the tour.

We need to select the next city in such a way-

$\text{cost}(i, S, j) = \min \text{cost}(i, S - \{i\}, j) + \text{dist}(i, j)$ where $i \in S$ and $i \neq j$



dist(i,j)	1	2	3	4
1	0	10	15	20
2	10	0	35	25
3	15	35	0	30
4	20	25	30	0



Let's see how our algorithm works:

Step 1)

We are considering our journey starting at city 1, visit other cities once and return to city 1.

Step 2)

S is the subset of cities. According to our algorithm, for all $|S| > 1$, we will set the distance $\text{cost}(i, S, 1) = \infty$.

Here $\text{cost}(i, S, j)$ means we are starting at city i , visiting the cities of S once, and now we are at city j .

We set this path cost as infinity because we do not know the distance yet. So the values will be the following:

$\text{Cost}(2, \{3, 4\}, 1) = \infty$; the notation denotes we are starting at city 2, going through cities 3, 4, and reaching 1. And the path cost is infinity.



Similarly-

$$\text{cost}(3, \{2, 4\}, 1) = \infty$$

$$\text{cost}(4, \{2, 3\}, 1) = \infty$$

Step 3) Now, for all subsets of S , we need to find the following:

$$\text{cost}(i, S, j) = \min \text{cost}(i, S - \{i\}, j) + \text{dist}(i, j), \text{ where } j \in S \text{ and } i \neq j$$

That means the minimum cost path for starting at i , going through the subset of cities once, and returning to city j .

Considering that the journey starts at city 1,

the optimal path cost would be $= \text{cost}(1, \{\text{other cities}\}, 1)$.





Warshall's Algorithm

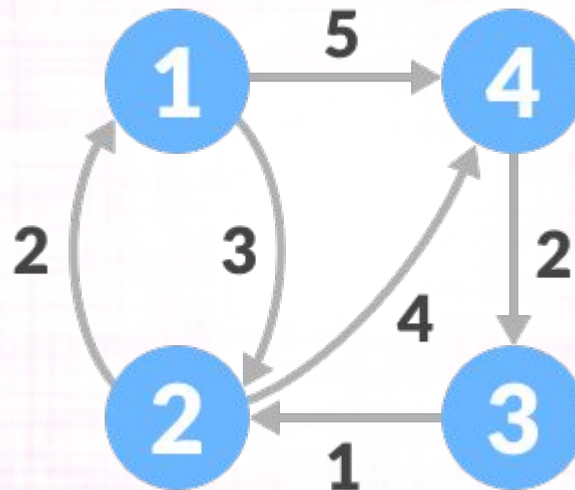
- The [Floyd Warshall Algorithm](#) is for solving all pairs of shortest-path problems.
- It is an algorithm for finding the shortest path between all the pairs of vertices in a weighted graph.
- This algorithm follows the dynamic programming approach to find the shortest path.
- Floyd-Warshall algorithm is also called as Floyd's algorithm, Roy-Floyd algorithm, Roy-Warshall algorithm, or WFI algorithm.





How Floyd-Warshall Algorithm Works?

Let the given graph be:





Follow the steps below to find the shortest path between all the pairs of vertices.

- 1) Create a matrix A^0 of dimension $n \times n$ where n is the number of vertices.
- 2) The row and the column are indexed as i & j respectively.
- 3) i & j are the vertices of the graph.
- 4) Each cell $A[i][j]$ is filled with the distance from the i th vertex to the j th vertex.
- 5) If there is no path from the i th vertex to the j th vertex, the cell is left as infinity.

$$A^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$



6) Now, create a matrix A^1 using matrix A^0 .

7) The elements in the first column and the first row are left as they are.

8) The remaining cells are filled in the following way.

9) Let k be the intermediate vertex in the shortest path from source to destination.

10) In this step k is the first vertex.

11) $A[i][j]$ is filled with

$(A[i][k] + A[k][j])$ if $(A[i][j] > A[i][k] + A[k][j])$

That is, if the direct distance from the source to the destination is greater than the path through the vertex k then the cell is filled with $A[i][k] + A[k][j]$.

12) In this step, k is vertex 1. We calculate the distance from source vertex to destination vertex through this vertex k .



$$A^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & 9 & 4 \\ \infty & 1 & 0 & 8 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Similarly, A^2 is created using A^1 . The elements in the second column and the second row are left as they are.

In this step k is the second vertex (i.e. vertex 2). The remaining steps are the same as mentioned above.

$$A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & & \\ 2 & 0 & 9 & 4 \\ & 1 & 0 & \\ \infty & & & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 9 & 5 \\ 2 & 0 & 9 & 4 \\ 3 & 1 & 0 & 5 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$



Similarly, A^3 and A^4 is also created.

$$A^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & & \infty & \\ & 0 & 9 & \\ \infty & 1 & 0 & 8 \\ & & 2 & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 9 & 5 \\ 2 & 0 & 9 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$

$$A^4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & & & 5 \\ & 0 & & 4 \\ & & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 7 & 5 \\ 2 & 0 & 6 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$

A^4 gives the shortest path between each pair of vertices.





■Floyd Warshall Algorithm Complexity

1)Time Complexity

There are three loops. Each loop has constant complexities. So, the time complexity of the Floyd-Warshall algorithm is $O(n^3)$

2)Space Complexity

The space complexity of the Floyd-Warshall algorithm is $O(n^2)$

■Floyd Warshall Algorithm Applications

- 1)To find the shortest path in a directed graph
- 2) To find the transitive closure of directed graphs
- 3)To find the Inversion of real matrices.





Assignment Questions

- 1) Write an algorithm for Knapsack problem using Greedy strategy.
- 2) Write an algorithm for Knapsack greedy problem. Find an optimal solution for following knapsack problem: $n=4$, $M=70$, $w = \{10, 20, 30, 40\}$, $P = \{20, 30, 40, 50\}$
- 3) State and explain the principle of dynamic programming. Name the elements of dynamic programming
- 4) Differentiate between dynamic programming and Greedy method.
- 5) Apply Warshall's Algorithm to find the transitive closure of the digraph defined by the following adjacency matrix.

0	1	0	0
0	0	1	0
0	0	0	1
0	0	0	0





- 6) Write the algorithm to compute 0/1 Knapsack problem using dynamic programming and explain it.
- 7) Explain the Travelling sales man problem .
- 8) Explain dynamic programming strategy? Enlist few applications which can be solved by using dynamic programming.
- 9) Explain activity selection problem in detail with example.
- 10) Explain Floyd's Algorithm for all pair shortest path algorithm with example and analyze its efficiency.



THANK YOU

